

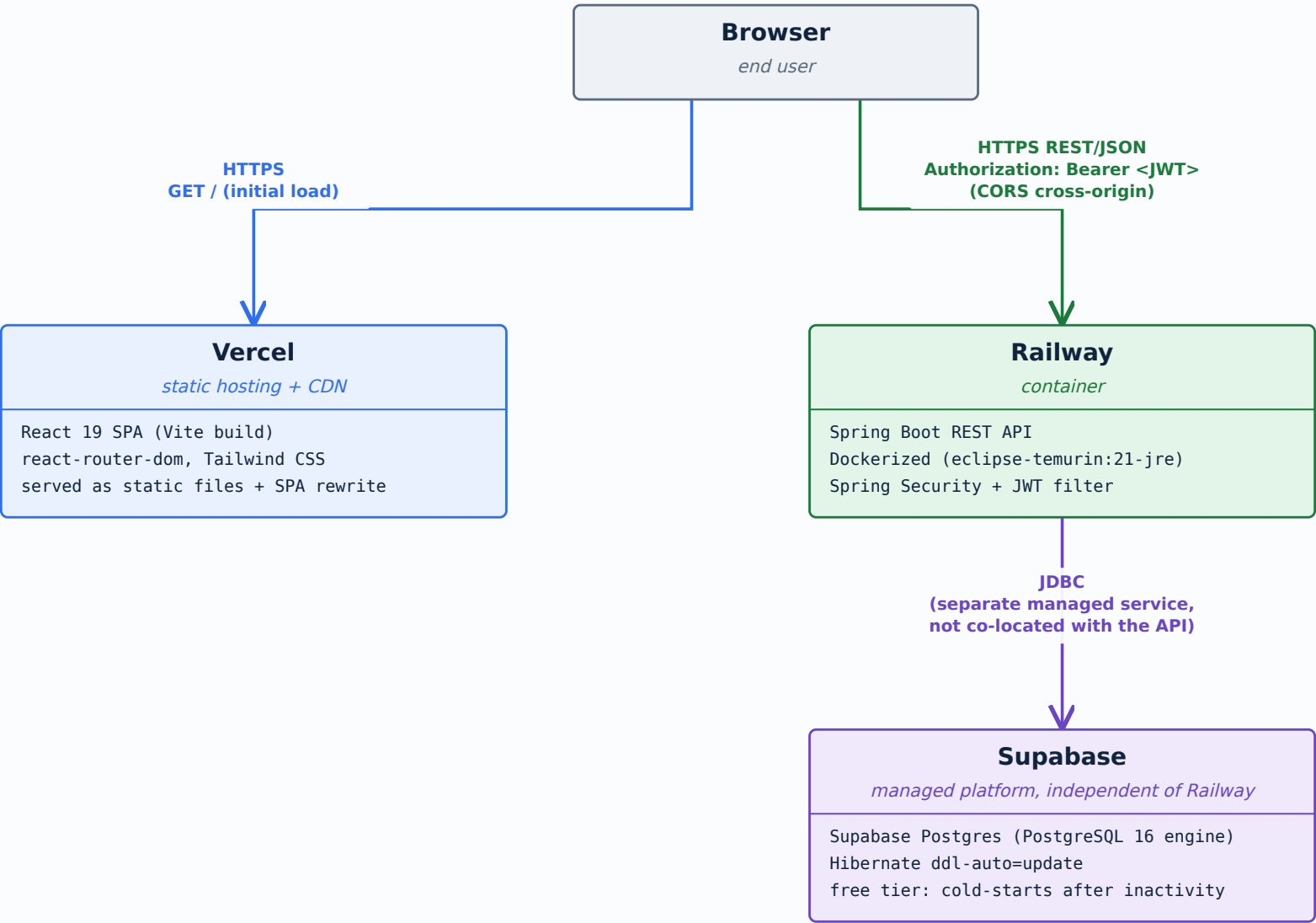
System Overview

Five complementary views of the system: how it's deployed, the backend domain model, the frontend's type/service layer, how the frontend and backend are structured internally, and the database schema behind it all.

-
- 1. **Architecture** — deployment topology: browser, Vercel, Railway, Supabase
 - 2. **Domain Class Diagram (Backend)** — the business objects and their relationships
 - 3. **Frontend Class Diagram** — TypeScript types and the service layer
 - 4. **Frontend & Backend** — internal layering of the React SPA and the Spring Boot API
 - 5. **Database** — the relational schema, same entities as the domain diagram, at the table level

1. Architecture

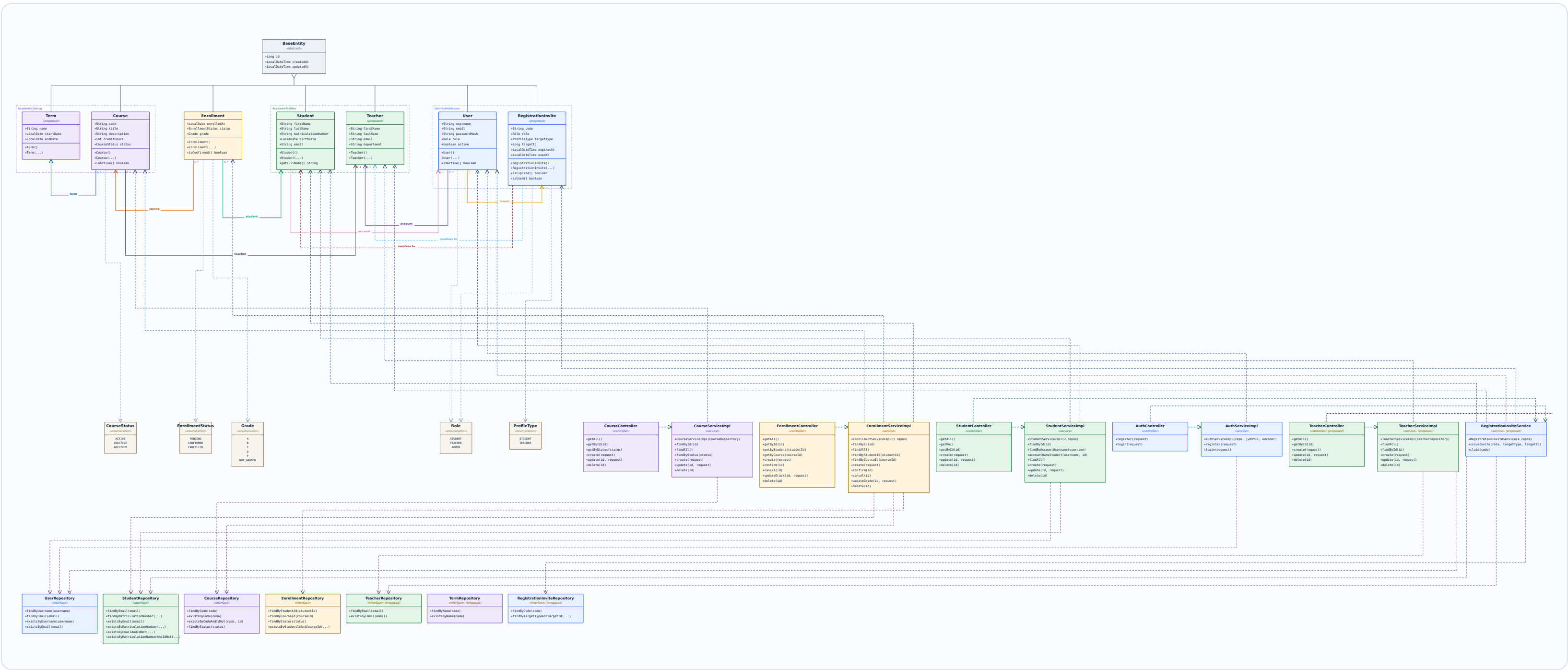
Production deployment topology — three independently deployed platforms, per the project README: the React SPA is static-hosted on **Vercel**; once loaded, it calls the Spring Boot API on **Railway** directly from the browser (cross-origin, hence the CORS configuration in **SecurityConfig**), authenticated via a JWT bearer token; the API in turn talks to a separately managed **Supabase** Postgres database over JDBC — Supabase is not part of the Railway deployment. Locally, the same three roles run instead via **docker-compose** (a local Postgres container + backend + an nginx-served frontend build), with none of the three cloud providers involved.



2. Domain Class Diagram (Backend)

Hand-routed orthogonal layout: one shared inheritance trunk into **BaseEntity**, each association/dependency in its own lane and its own color for traceability, correct UML multiplicities, and constructors on every entity. Below the entity row the same technique repeats three times: the five «enumeration» classes as dependency targets; six «service» classes (methods and constructor read from the actual **ServiceImpl* code) as dependency sources routed up into the entities they use; and seven «interface» Spring Data repositories one tier deeper, as the dependency targets each service's constructor injects. Each «controller» sits directly left of its one service (methods read from the actual **Controller* code); three also reach across to the proposed **RegistrationInviteService** for issuing/claiming an invite. A «proposed» tag marks classes, controllers, services, and repositories not yet in the codebase.

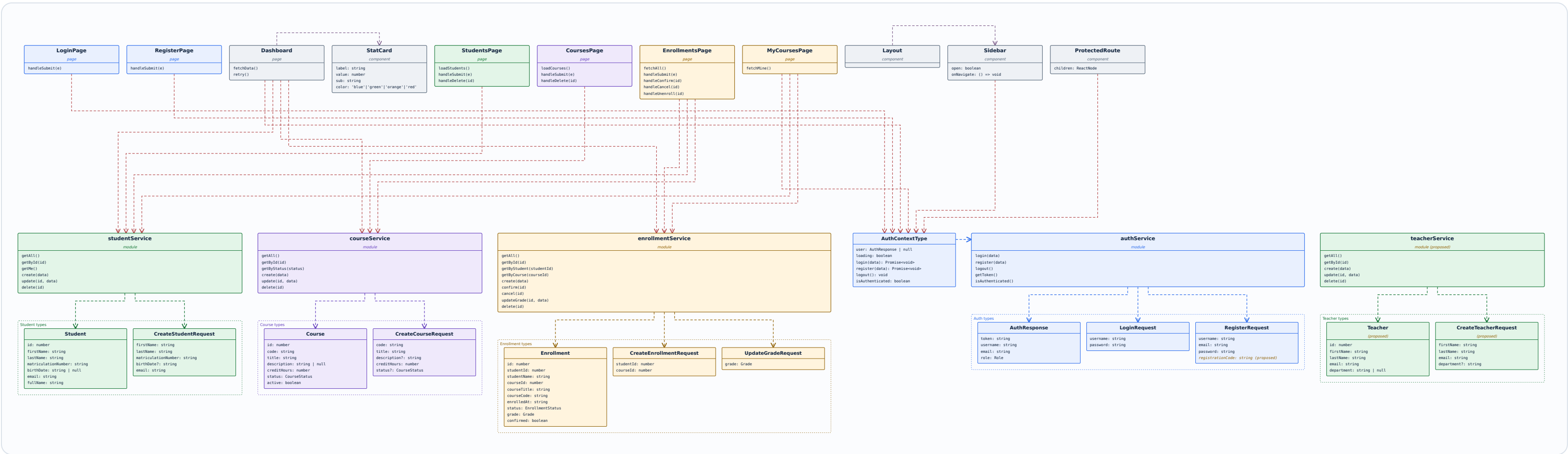
Base / audit Identity & access Academic profiles Academic catalog Association (Enrollment) Enumeration



Student Manager App · Domain Class Diagram (Backend)

3. Frontend Class Diagram

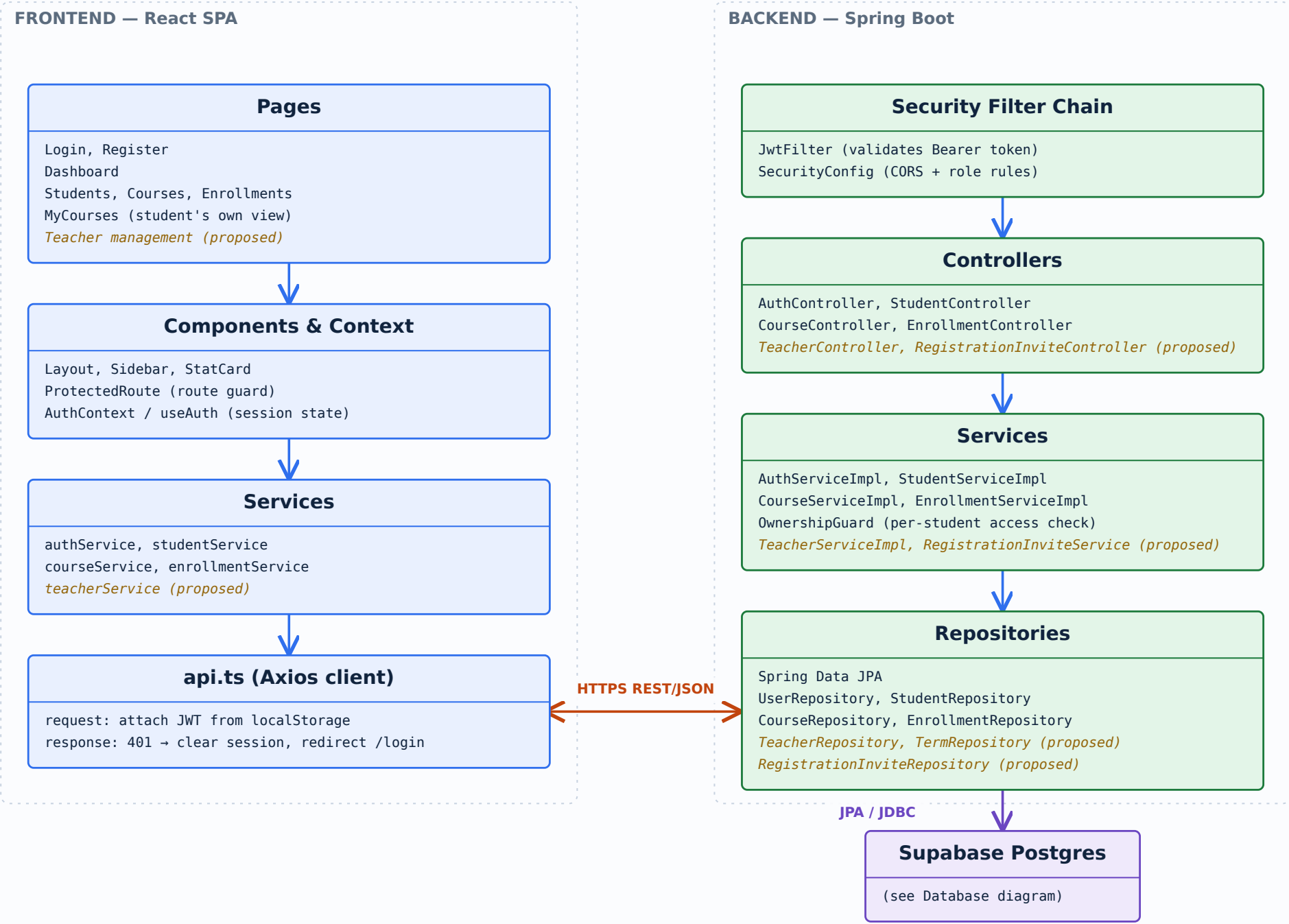
The frontend has no OOP classes (React components are functions), so this models what actually is static structure, top to bottom: the 7 route-level «page» components and 4 reusable «component»s (their real internal handler names or props stand in for methods/fields), the Axios service modules and `AuthContextType` they call into, and the TypeScript interface types those services send/receive (grouped by domain concept, each paired with its `Create*Request` write-shape where one exists). Red dashed arrows are a page/component's dependency on a service or on `AuthContextType`; the short blue arrow is `AuthContextType`'s `login/ register` calling into `authService`; the two black dashed hops (`Layout`→`Sidebar`, `Dashboard`→`StatCard`) are same-row component composition. Includes the proposed `teacherService` and `Teacher` types alongside the existing frontend.



Student Manager App · Frontend Class Diagram

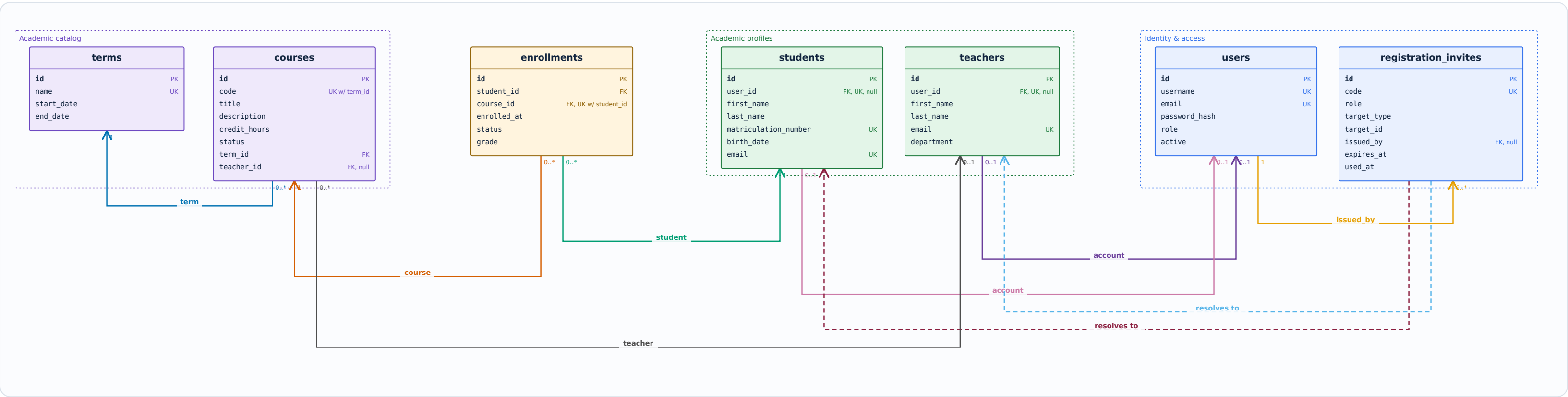
4. Frontend & Backend

Internal layering on each side of the REST boundary. The frontend's `api.ts` Axios client attaches the JWT to every request and redirects to `/login` on a 401 (except on the auth endpoints themselves, where a 401 just means bad credentials). On the backend, every request passes through `JwtFilter` before reaching a controller; `OwnershipGuard` additionally restricts a `STUDENT` to their own data on the enrollment endpoints. Items in amber italic (e.g. `TeacherController`, `RegistrationInviteService`) are part of the proposed redesign and don't exist in the codebase yet — kept here for consistency with the domain and database diagrams.



5. Database

Same seven entities and relationships as the domain diagram, at the table level: column names, primary/foreign/unique keys, and nullability instead of Java types and methods. This is the proposed redesign's schema (adds teachers, terms, and registration_invites to fix the fragile email-matching account link) — see the companion database design PDF for full column definitions and sample data.



Student Manager App · Database